Assignment 7

Mayur Zope SE Comp A 75

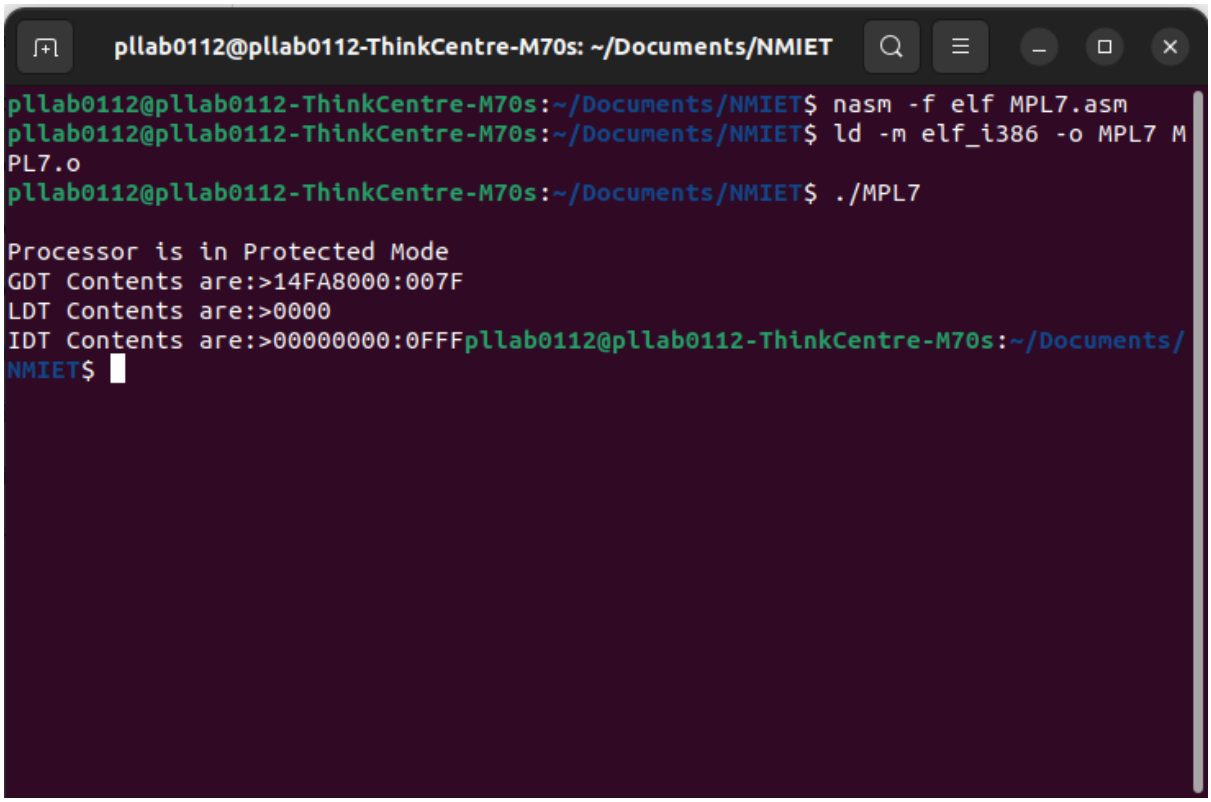
;This program first check the mode of processor(Real or Protected),

;then reads GDTR, LDTR and IDTR and displays the same.

```
%macro IO 4
  mov eax,%1
  mov ebx,%2
  mov ecx,%3
  mov edx,%4
  int 80h
%endmacro

section .data
  gdtmsg db 10,"GDT Contents are:>"
  gmsg_len equ $-gdtmsg
  ldtmsg db 10,"LDT Contents are:>"
  lmsg_len equ $-ldtmsg
  idtmsg db 10,"IDT Contents are:>"
  imsg_len equ $-idtmsg
  nxline db 10
  colon db ':'
  rmodemsg db 10,"Processor is in Real Mode"
  rmsg_len equ $-rmodemsg
  pmodemsg db 10,"Processor is in Protected Mode"
  pmsg_len equ $-pmodemsg

section .bss
  gdt resd 1
      resw 1
  ldt resw 1
  idt resd 1
      resw 1
  output resb 04
```

```
  cr0_data resd 1


section .text
global _start
_start:
  smsw eax            ;Reading CR0
  mov [cr0_data],eax  ;store the msw into local variable
  ror eax,1           ;Checking PE bit, if 1=Protected Mode, else Real
Mode
  jc prmode           ;if carry set its protected mode
  IO 4,1, rmodemsg,rmsg_len ;else real mode
  jmp next
prmode:
  IO 4,1, pmodemsg,pmsg_len


next:
  sgdt [gdt]          ;read gdt into local variable
  sldt [ldt]          ;read ldt into local variable
  sidt [idt]          ;read idt into local variable

  IO 4,1, gdtmsg,gmsg_len
  mov bx,[gdt+4]
  call display
  mov bx,[gdt+2]
  call display
  IO 4,1, colon,1
  mov bx,[gdt]
  call display

  IO 4,1, ldtmsg,lmsg_len
  mov bx,[ldt]
  call display
```

```
  IO 4,1, idtmsg,imsg_len
  mov bx,[idt+4]
  call display
  mov bx,[idt+2]
  call display

  IO 4,1, colon,1
  mov bx,[idt]
  call display

exit:
  mov eax,01
  mov ebx,00
  int 80h

display:
  mov esi,output    ;point esi to buffer
  mov ch,04         ;load number of digits to display
  mov cl,04         ;load count of rotation in cl
up1:
  rol bx,cl         ;rotate number left by four bits
  mov dl,bl         ;move lower byte in dl
  and dl,0fh        ;mask upper digit of byte in dl
  cmp dl,09h        ;compare with 39h
  jbe skip1         ;if less than 39h akip adding 07 more
  add dl,07h        ;else add 07
skip1:
  add dl,30h        ;add 30h to calculate ASCII code
  mov [esi],dl      ;store ASCII code in buffer
  inc esi           ;point to next byte
  dec ch            ;decrement the count of digits to display
```

```
  jnz up1            ;if not zero jump to repeat
  IO 4, 1, output, 4
Ret
```

// OUTPUT